React

Table of Contents

React Fundamentals

Instalación con Vite

```text
npm create vite@latest
```

Project name: my-app Select a framework: **React** Select a variant: **JavaScript + SWC** (SWC es un compilador JS/TS rápido escrito en Rust)

```text
cd my-app
npm install
npm run dev
```

Recommended tools

Browser extensions: React Developer Tools, Redux Dev Tools

VSCode extensions: TypeScript Importer, Simple React Snippets, ES7+ React/Redux/React-Native snippets

React Library

React is a JavaScript library used to build single-page applications (SPAs) by creating reusable UI components.

Single Page Application (SPA)

Refers to a web application that dynamically rewrites the current page rather than loading entire new pages from the server.

Pros:

- Much faster user experience (no full page reloads)
- Reduces server resource consumption
- Same API can be reused for mobile apps
- Easy to transform into Progressive Web Apps (offline support)
- Backend and frontend teams can work independently

Cons:

- Not ideal for SEO-required apps (unless using SSR)

- Large JavaScript bundles
- Potential memory leaks (app runs for extended periods)
- Poor performance on low-power devices
- Accessibility concerns if JS is disabled

Overriding Navigation

As default browser navigation is eliminated in SPAs, URLs must be manually managed via a **router** (e.g., React Router).

Why React is not a framework

React is a library because it needs additional dependencies to build a complete web application, whereas a framework includes tools/dependencies out of the box.

How does React work?

React operates by creating an in-memory **virtual DOM** rather than directly manipulating the browser's DOM.

Features of React

- **Declarative** — describe what the UI should look like, not how to achieve it
- **Server-Side Rendering (SSR)** — renders components on the server (e.g., Next.js)
- **Component-Based Architecture** — break UI into reusable, self-contained components
- **JSX** — syntax extension that allows writing HTML-like code within JavaScript
- **Virtual DOM** — lightweight in-memory representation of the real DOM
- **One-way Data Binding** — data flows from parent to child via props

Declarative vs Imperative

JSX

```jsx
// Declarative
const element = <h1>Hello, world</h1>;

// Imperative
const element = document.createElement('h1');
element.innerHTML = 'Hello, world';
```

React Element

The smallest building block of a React application. Represents a virtual DOM node. Elements are **immutable**.

```jsx
const element = <h1>Hello, world!</h1>;
```

Is equivalent to:

```js
const element = React.createElement('h1', null, 'Hello, world!');
```

Rendering Elements

```jsx
const element = <h1>Hello, world!</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(element);
```

React Element vs Component

Component: A function (or class) that returns element(s). **Element:** What React renders.

Virtual DOM

A lightweight copy of the browser's DOM used to optimize performance by reducing direct manipulation of the real DOM.

Why Virtual DOM?

- Real DOM manipulation is slow and expensive
- Virtual DOM allows React to batch updates and minimize changes to the real DOM

How it works

1. **Initial Render** — React creates a Virtual DOM tree
2. **Re-render** — on state/prop changes, React creates a new Virtual DOM tree
3. **Diffing (Reconciliation)** — React compares the new tree with the previous one
4. **Minimal Updates** — only necessary changes are applied to the real DOM

Key Benefits

- **Performance** — minimizes direct DOM manipulation
- **Batching** — groups multiple updates together
- **Declarative UI** — describe desired state, React handles updates

React.StrictMode

A development-only tool that detects problems in the application.

- **Double renders** — components render twice in dev to detect side effects
- **Detects unsafe lifecycle methods** — warns about deprecated APIs
- **Warns about string refs** — recommends `createRef` or `useRef`
- **Detects unexpected side effects** — helps find impure rendering logic
- **Helps migrate to future React features** — prepares for upcoming changes

StrictMode has no effect in production. It only runs in development.

JSX

```jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';

createRoot(document.getElementById('root')).render(
    <StrictMode>
        <App />
    </StrictMode>
);
```

Estructura de archivos de un proyecto

TEXT

```
src/
  components/     Componentes reutilizables
  hooks/          Custom hooks
  utils/          Funciones helper, utilidades
  services/       Llamadas a APIs, lógica externa
  assets/         Imágenes, fuentes, iconos
  styles/         Archivos CSS globales
  App.jsx         Componente raíz
  main.jsx        Punto de entrada (monta React en el DOM)
public/
  index.html      HTML base
  data/           Archivos JSON accesibles por URL
vite.config.*     Configuración de Vite
package.json      Dependencias y scripts
```

JSX

JSX (JavaScript XML) is a syntax extension for JavaScript that allows writing HTML-like code inside JavaScript. It's not valid JavaScript on its own — tools like Babel or SWC transpile it into regular `React.createElement()` calls.

JSX

```jsx
// JSX
const element = <h1>Hello, world!</h1>;

// What React compiles it to
const element = React.createElement('h1', null, 'Hello, world!');
```

JSX makes it easier to visualize and write UI structures compared to pure JavaScript calls.

JSX Expressions

Use curly braces `{}` to embed any JavaScript expression inside JSX.

Variables:

JSX

```jsx
const name = "Alan";
const element = <h1>Hello, {name}</h1>;
```

Function calls:

JSX

```jsx
function getGreeting() {
    return "Welcome back!";
}

const element = <h1>{getGreeting()}</h1>;
```

Template literals:

JSX

```jsx
const user = "Alan";
const element = <p>Hello, {`Mr. ${user}`}</p>;
```

Arithmetic:

```jsx
const element = <p>{2 + 3}</p>;
```

Object access:

```jsx
const user = { name: "Alan", age: 25 };
const element = <p>{user.name} is {user.age}</p>;
```

> You can only embed **expressions**, not statements. `{if (true) {}}` won't work.

JSX Rules

1. Return a single root element

```jsx
<div>
    <h1>Hedy Lamarr's Todos</h1>
    <img
        src="https://i.imgur.com/yXOvdOSs.jpg"
        alt="Hedy Lamarr"
        class="photo"
    />
    <ul>
        ...
    </ul>
</div>
```

Fragment

If you don't want to add an extra `<div>`, use `<>` `</>` or `<Fragment>`:

JSX

```jsx
<>
    <h1>Hedy Lamarr's Todos</h1>
    <img
        src="https://i.imgur.com/yXOvdOSs.jpg"
        alt="Hedy Lamarr"
        class="photo"
    />
    <ul>
        ...
    </ul>
</>
```

JSX

```jsx
import { Fragment } from "react";

export default function Component() {
    return (
        <Fragment>
            <h1>Hedy Lamarr's Todos</h1>
            <img
                src="https://i.imgur.com/yXOvdOSs.jpg"
                alt="Hedy Lamarr"
                class="photo"
            />
            <ul>
                ...
            </ul>
        </Fragment>
    );
}
```

2. Close all tags

JSX requires explicit closing. HTML void elements must be self-closed with `/>`.

JSX

```jsx
// Bad — won't compile
<img src="photo.jpg">
<br>
<input type="text">

// Good — self-closed
<img src="photo.jpg" />
<br />
<input type="text" />
```

Elements with children use opening and closing tags normally:

JSX

```jsx
<li>Invent new traffic lights</li>
```

3. Conditional Rendering

if/else:

JSX

```jsx
if (isPacked) {
    return <li className="item">{name}</li>;
}
return <li className="item">{name}</li>;
```

Ternary operator:

JSX

```jsx
return (
    <li className="item">
        {isPacked ? name + 'John' : name}
    </li>
);
```

Logical AND (&&):

JSX

```jsx
return (
    <li className="item">
        {name} {isPacked && 'hola'}
    </li>
);
```

> Don't put numbers on the left side of `&&`. `0 && <p>hi</p>` renders `0`. Fix: `messageCount > 0 && <p>New messages</p>`.

4. Lists and Keys

Use `.map()` to render arrays as JSX elements.

JSX

```jsx
const numbers = [1, 2, 3, 4, 5];
const listItems = numbers.map((number) =>
    <li key={number.toString()}>{number}</li>
);
const element = <ul>{listItems}</ul>;
```

> Each element in a list needs a unique `key` prop. See Components & Props - Key Prop for details.

5. Use camelCase for HTML attributes

JSX

```jsx
// class → className
<div className="container"></div>

// for → htmlFor
<label htmlFor="inputId">Input:</label>

// Events
<button onClick={handleClick}>Click me</button>
```

6. Components vs HTML elements

JSX distinguishes between HTML elements and React components by **case**:

- **Lowercase** — treated as HTML: `<div>`, `<span>`, `<img>`

- **Uppercase** — treated as a component: `<MyComponent />`

JSX

```jsx
// HTML element
<div className="container">Hello</div>

// React component (must be defined or imported)
function Greeting({ name }) {
    return <h1>Hello, {name}</h1>;
}

// Usage
<Greeting name="Alan" />
```

> If a component starts with a lowercase letter, React treats it as an HTML tag and it won't work. Always start component names with an uppercase letter.

Components & Props

Component

A reusable, self-contained building block that defines a part of the UI. Components are like JS functions/classes that accept **props** and return React elements (JSX).

JSX

```jsx
export default function Profile() {
    return (
        <img
            src="https://i.imgur.com/MK3eW3Am.jpg"
            alt="Katherine Johnson"
        />
    );
}
```

> Always start component names with a capital letter. `<div />` is an HTML tag, but `<Greeting />` is a component.

Types of Components

- **Functional Components** — plain JS functions that accept props and return JSX
- **Class Components** — ES6 classes extending `React.Component` with lifecycle methods
- **Stateful vs Stateless** — stateful manages internal state; stateless only receives props
- **Presentational vs Container** — presentational focuses on UI; container handles logic/state

Why Components?

- **Modularity** — break UI into smaller, self-contained units
- **Reusability** — use the same component across different parts of the app
- **Separation of Concerns** — separate logic from presentation

Props

Props (short for "properties") pass data from parent to child. They are **read-only** and immutable within the child.

JSX

```jsx
function ParentComponent() {
    return <ChildComponent name="John" age={25} />;
}

function ChildComponent(props) {
    return (
        <div>
            <p>Name: {props.name}</p>
            <p>Age: {props.age}</p>
        </div>
    );
}
```

Destructuring Props

Extract props directly in the function parameter instead of accessing via `props.`:

JSX

```jsx
// Without destructuring
function ChildComponent(props) {
    return <p>{props.name}</p>;
}

// With destructuring
function ChildComponent({ name, age }) {
    return (
        <div>
            <p>Name: {name}</p>
            <p>Age: {age}</p>
        </div>
    );
}
```

You can also rename during destructuring:

JSX

```jsx
function ChildComponent({ name: userName }) {
    return <p>{userName}</p>;
}
```

Default Props

Set fallback values when a prop is not provided:

```jsx
function Greeting({ name = "Guest", age = 0 }) {
    return (
        <p>Hello, {name}. You are {age} years old.</p>
    );
}

<Greeting />               // "Hello, Guest. You are 0 years old."
<Greeting name="Alan" />   // "Hello, Alan. You are 0 years old."
```

Props vs State

Props	State
Passed from parent to child	Managed internally within component
Immutable	Can change over time
Read-only in child	Updated via setter (e.g., setState)

Children Prop

`props.children` passes content between opening and closing tags of a component.

```jsx
function ParentComponent() {
    return (
        <ChildComponent>
            <p>This is the children content.</p>
        </ChildComponent>
    );
}

function ChildComponent({ children }) {
    return <div>{children}</div>;
}
```

Key Prop

`key` is a special prop used by React to identify which items in a list have changed, been added, or removed. It's not passed to the component — React uses it internally.

Improves performance by avoiding unnecessary re-renders.

```jsx
items.map((item) => <li key={item.id}>{item.name}</li>)
```

Rules for Keys

- Must be **unique** among siblings
- Must be **stable** — don't use `Math.random()` or `Date.now()`
- Use IDs from your data when available

```jsx
// Good — unique ID from data
items.map((item) => <li key={item.id}>{item.name}</li>)

// Bad — changes every render
items.map((item) => <li key={Math.random()}>{item.name}</li>)
```

Don't Use Index as Key

```jsx
// Bad — causes issues with reordering, filtering, adding/removing
items.map((item, index) => <li key={index}>{item.name}</li>)

// Good
items.map((item) => <li key={item.id}>{item.name}</li>)
```

Using array index as key causes bugs when:

- Items are reordered
- Items are inserted or deleted
- Items have local state

Props Drilling

Passing data through multiple component layers even when some don't need it.

See Lifting State Up — sharing state by moving it to the closest common parent.

If lifting becomes impractical (3+ levels deep), then use:

- **Context API** — global state without prop drilling
- **State Management Libraries** (Redux, Zustand, etc.)

Composition

React recommends composition over inheritance. Build complex UIs by nesting components inside others.

Using children:

JSX

```jsx
function Card({ children }) {
    return <div className="card">{children}</div>;
}

function App() {
    return (
        <Card>
            <h2>Title</h2>
            <p>Content</p>
        </Card>
    );
}
```

Using custom props for slots:

JSX

```jsx
function Layout({ sidebar, content }) {
    return (
        <div className="layout">
            <aside>{sidebar}</aside>
            <main>{content}</main>
        </div>
    );
}

function App() {
    return (
        <Layout
            sidebar={<Navigation />}
            content={<HomePage />}
        />
    );
}
```

PropTypes

Runtime type-checking for props. Optional but recommended.

TEXT

```
npm install prop-types
```

JSX

```jsx
import PropTypes from 'prop-types';

function MyComponent({ name, age, isStudent }) {
    return (
        <div>
            <p>Name: {name}</p>
            <p>Age: {age}</p>
            <p>Is Student: {isStudent ? 'Yes' : 'No'}</p>
        </div>
    );
}

MyComponent.propTypes = {
    name: PropTypes.string.isRequired,
    age: PropTypes.number,
    isStudent: PropTypes.bool,
};
```

Lifting State Up

The Problem

When two sibling components need to share the same data, neither can own the state independently. One component updates the data, but the other doesn't see the change.

JSX

```jsx
// ❌ Broken: each component owns its own state
function SearchInput() {
    const [query, setQuery] = useState('');
    return (
        <input
            value={query}
            onChange={e => setQuery(e.target.value)}
            placeholder="Search..."
        />
    );
}

function FilteredList() {
    const [query, setQuery] = useState(''); // independent state!
    const items = ['Apple', 'Banana', 'Cherry', 'Date'];
    const filtered = items.filter(item =>
        item.toLowerCase().includes(query.toLowerCase())
    );
    return (
        <ul>
            {filtered.map(item => <li key={item}>{item}</li>)}
        </ul>
    );
}

function App() {
    return (
        <div>
            <SearchInput />
            <FilteredList />
        </div>
    );
}
// Problem: typing in SearchInput doesn't filter FilteredList
// because each has its own `query` state
```

The Solution

Move the shared state up to the **closest common parent**. The parent owns the state and passes it down via props.

JSX

```jsx
// ✅ Correct: parent owns the state
function App() {
    const [query, setQuery] = useState('');

    return (
        <div>
            <SearchInput query={query} onChange={setQuery} />
            <FilteredList query={query} />
        </div>
    );
}

function SearchInput({ query, onChange }) {
    return (
        <input
            value={query}
            onChange={e => onChange(e.target.value)}
            placeholder="Search..."
        />
    );
}

function FilteredList({ query }) {
    const items = ['Apple', 'Banana', 'Cherry', 'Date'];
    const filtered = items.filter(item =>
        item.toLowerCase().includes(query.toLowerCase())
    );
    return (
        <ul>
            {filtered.map(item => <li key={item}>{item}</li>)}
        </ul>
    );
}
```

> The state lives in `App`. Both children receive it via props. When you type in the input, the parent updates and the list filters automatically.

How It Works

```text
App (owns state: query="")
 ├─ SearchInput (receives query + onChange via props)
 └─ FilteredList (receives query via props)
```

1. State is declared in the common parent
2. Downward flow: parent passes state as props to children
3. Upward flow: children call the parent's setter to update
4. Parent re-renders → both children re-render with new data

Rules

- **State lives where it's shared.** If two components need the same value, lift it to their closest common ancestor.
- **Single source of truth.** One component owns the state; others just read it.
- **Keep state minimal.** Only store what you need. Derive the rest (e.g., filtered list from query + items).

Lifting State vs Context

Pattern	When to use
Lifting State Up	2-3 sibling components share state through a common parent
Context API	Data needed by many components across different branches of the tree

Lifting State Up is usually sufficient. Context is for when prop passing becomes unwieldy (3+ levels deep).

Styling in React

Importing CSS Files

Import a CSS file directly into a component. Vite (and other bundlers) process it and inject it into the page.

JSX

```jsx
import './App.css';

function App() {
    return <div className="container">Hello</div>;
}
```

The class names are **global** — any CSS file you import applies to the entire app. This can cause naming conflicts in larger projects.

Inline Styles

Styles are passed directly as JavaScript objects. Properties are in camelCase.

JSX

```jsx
const divStyle = {
    color: 'blue',
    backgroundColor: 'lightgray',
};

const element = <div style={divStyle}>Styled text</div>;
```

Full Examples

JSX

```jsx
function InlineStyleInline() {
    return (
        <button
            style={{
                backgroundColor: "purple",
                color: "white",
                padding: "10px 20px",
                border: "none",
                borderRadius: "5px",
                cursor: "pointer",
            }}
        >
            ¡Haz clic aquí!
        </button>
    );
}
```

JSX

```jsx
function InlineStyleExample() {
    const style = {
        backgroundColor: "lightblue",
        color: "darkblue",
        padding: "10px",
        borderRadius: "5px",
    };

    return (
        <div style={style}>
            ¡Hola! Este es un ejemplo de estilos inline en React.
        </div>
    );
}
```

When to use inline styles:

- Truly dynamic values (calculated at runtime)
- Quick one-off styling
- Avoid for static styles — use CSS files or Modules instead

Limitations:

- No pseudo-selectors (`:hover` , `:focus`)
- No media queries
- No keyframes animations

For these, use regular CSS files or CSS Modules.

CSS Modules

CSS Modules scope class names locally by default. The file must end with `.module.css`.

CSS

```css
/* Button.module.css */
.primary {
    background-color: blue;
    color: white;
}

.secondary {
    background-color: gray;
    color: white;
}
```

JSX

```jsx
import styles from './Button.module.css';

function Button({ variant }) {
    return (
        <button className={styles[variant]}>
            Click me
        </button>
    );
}
```

The `styles` object maps original class names to unique, hashed versions (`Button_primary_a1b2c`). No naming conflicts.

Conditional className

Apply classes dynamically based on state or props.

Ternary:

JSX

```jsx
<button className={isActive ? 'btn active' : 'btn'}>
    Click
</button>
```

Logical OR:

JSX

```jsx
<div className={error || 'default-class'}>
    Message
</div>
```

Template literal:

JSX

```jsx
<div className={`card ${isLarge ? 'card--large' : ''}`}>
    Content
</div>
```

With CSS Modules:

JSX

```jsx
import styles from './Card.module.css';

<div className={`${styles.card} ${isLarge ? styles.large : ''}`}>
    Content
</div>
```

Dynamic Styles

Use JavaScript objects when styles depend on state or props. Properties are in camelCase.

JSX

```jsx
function ProgressBar({ percent }) {
    return (
        <div
            style={{
                width: '100%',
                height: '20px',
                backgroundColor: '#eee',
            }}
        >
            <div
                style={{
                    width: `${percent}%`,
                    height: '100%',
                    backgroundColor: 'blue',
                    transition: 'width 0.3s ease',
                }}
            />
        </div>
    );
}
```

Use **inline styles** for truly dynamic values (calculated at runtime). Use **className** for static styles.

Why camelCase

CSS properties with dashes are converted to camelCase in JavaScript:

CSS	JSX
`background-color`	`backgroundColor`
`font-size`	`fontSize`
`border-radius`	`borderRadius`
`text-align`	`textAlign`

CSS Variables in React

Pass dynamic values from JavaScript to CSS using custom properties:

JSX

```jsx
function ThemeButton({ color }) {
    return (
        <button style={{ '--btn-color': color }}>
            Themed Button
        </button>
    );
}
```

CSS

```css
/* Button.css */
button {
    background-color: var(--btn-color);
    color: white;
    padding: 10px 20px;
}
```

This is useful when you need to set CSS values that can't be expressed with inline styles (pseudo-selectors, media queries).

Tailwind CSS

A utility-first CSS framework. Instead of writing CSS files, you apply small utility classes directly in JSX.

Installation with Vite

BASH

```bash
npm install -D tailwindcss @tailwindcss/vite
```

JS

```js
// vite.config.js
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
    plugins: [tailwindcss()],
})
```

CSS

```css
/* index.css */
@import "tailwindcss";
```

Basic Usage

JSX

```jsx
<button className="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue
    Click me
</button>
```

Common Utilities

Category	Classes	Example
Spacing	`p-*`, `m-*`, `px-*`, `py-*`	`p-4`, `mx-auto`, `mt-2`
Colors	`bg-*`, `text-*`, `border-*`	`bg-red-500`, `text-gray-700`
Typography	`text-sm`, `text-lg`, `font-bold`	`text-xl font-semibold`
Flexbox	`flex`, `items-center`, `justify-between`	`flex gap-4`
Grid	`grid`, `grid-cols-3`, `col-span-2`	`grid grid-cols-3 gap-4`
Sizing	`w-*`, `h-*`, `max-w-*`	`w-full`, `h-screen`
Rounded	`rounded`, `rounded-lg`, `rounded-full`	`rounded-lg`

Responsive Design

Use breakpoint prefixes to apply styles at specific screen sizes:

JSX

```jsx
<div className="text-sm md:text-lg lg:text-xl">
    Responsive text
</div>
```

Prefix	Min-width
`sm:`	640px
`md:`	768px
`lg:`	1024px
`xl:`	1280px

Dark Mode

Use the `dark:` prefix with class strategy:

JSX

```jsx
<div className="bg-white dark:bg-gray-900 text-black dark:text-white">
    Theme content
</div>
```

Conditional Classes

JSX

```jsx
// Template literal
className={`px-4 py-2 ${isActive ? 'bg-blue-500' : 'bg-gray-300'}`}

// With clsx library (optional)
import clsx from 'clsx';
className={clsx('px-4 py-2', isActive && 'bg-blue-500')}
```

Security in React

XSS Prevention

By default, React DOM escapes any values embedded in JSX before rendering, preventing injection attacks. Everything is converted to a string before rendering.

JSX

```jsx
const title = "<script>alert('hacked')</script>";

// React renders this as plain text, not as executable HTML
const element = <h1>{title}</h1>;
// Output: &lt;script&gt;alert('hacked')&lt;/script&gt;
```

dangerouslySetInnerHTML

React provides `dangerouslySetInnerHTML` to render raw HTML. Use it only with trusted content — never with user input.

JSX

```jsx
function Markup({ html }) {
    return <div dangerouslySetInnerHTML={{ __html: html }} />;
}
```

> This bypasses React's XSS protection. Always sanitize HTML before rendering it.

Sanitize User Input

Never trust data from users, APIs, or URLs. Always sanitize before rendering.

JSX

```jsx
// Bad — renders raw user input
<div>{userInput}</div>

// Good — React escapes it automatically
// But be careful with dangerouslySetInnerHTML
```

When you need to render HTML from user input, use a sanitizer like DOMPurify.

DOMPurify

The standard library for sanitizing HTML. Strips dangerous tags and attributes.

```bash
npm install dompurify
```

```jsx
import DOMPurify from 'dompurify';

function SafeMarkup({ html }) {
    const clean = DOMPurify.sanitize(html);

    return <div dangerouslySetInnerHTML={{ __html: clean }} />;
}
```

Always use DOMPurify when rendering user-provided HTML, even with `dangerouslySetInnerHTML`.

URL Safety

Never use user-provided URLs directly in `href` or `src` without validation. Malicious protocols can execute code.

```jsx
// Bad — allows javascript: protocol
<a href={userUrl}>Click</a>

// Good — validate URL protocol
function SafeLink({ url, children }) {
    const isSafe = /^https?:\/\//.test(url);

    if (!isSafe) return <span>{children}</span>;

    return <a href={url} target="_blank" rel="noopener noreferrer">{chilc
}
```

Also use `rel="noopener noreferrer"` on external links to prevent tabnapping.

eval() and Dangerous Patterns

Never use these in React (or any frontend code):

JSX

```jsx
// Bad — executes arbitrary code
eval(userInput);
new Function(userInput)();
setTimeout("alert('hacked')", 1000);
setInterval("doSomething()", 1000);
```

Use function references instead:

JSX

```jsx
// Good
setTimeout(() => alert('safe'), 1000);
```

Never Store Secrets

Never put sensitive data in client-side code. Everything in the browser is visible to users.

JSX

```jsx
// Bad — exposed in browser
const API_KEY = 'sk-1234567890';
const SECRET_TOKEN = 'abc123';

// Good — use environment variables (still visible, but less obvious)
const API_KEY = import.meta.env.VITE_API_KEY;

// Better — keep secrets on the backend
// Frontend calls your API, your API calls external services
```

Environment variables (`import.meta.env.VITE_*`) are still embedded in the build output and visible in browser DevTools. The only truly safe place for secrets is on your backend.

React Hooks

Hooks allow functional components to use state and lifecycle features that were previously only available in class components. Hook names start with `use`.

Rules of Hooks

1. **Only call Hooks at the top level** — before return, not inside loops, conditions, nested functions, or try/catch/finally.
2. **Only call Hooks from React functions** — from React function components or custom Hooks.

Available Hooks

Hook	Description	File
`useState`	Add state to functional components	State Hooks
`useReducer`	Complex state logic with reducer pattern	State Hooks
`useEffect`	Side effects (data fetching, subscriptions, DOM manipulation)	Effect Hooks
`useLayoutEffect`	Synchronous side effects (DOM measurements, preventing flicker)	Effect Hooks
`useRef`	Mutable values that persist across renders	Refs & Memo
`useCallback`	Memoize functions (prevent unnecessary re-renders)	Refs & Memo
`useMemo`	Memoize computed values (expensive calculations)	Refs & Memo
`useContext`	Consume React Context values	Advanced Hooks
Custom Hooks	Reusable logic encapsulation	Advanced Hooks

Quick Guide

- **Need state?** → `useState` or `useReducer` (for complex logic)
- **Need side effects?** → `useEffect` (or `useLayoutEffect` for DOM measurements)
- **Need mutable values?** → `useRef`
- **Need to memoize?** → `useCallback` (functions) or `useMemo` (values)
- **Need context?** → `useContext`
- **Need reusable logic?** → Custom Hooks

React Hooks: State

Hooks allow functional components to use state and lifecycle features that were previously only available in class components. Hook names start with `use`.

Rules of Hooks

1. **Only call Hooks at the top level** — before return, not inside loops, conditions, nested functions, or try/catch/finally.
2. **Only call Hooks from React functions** — from React function components or custom Hooks.

useState

Adds state to functional components. Returns an array with the current state value and a function to update it.

JSX

```jsx
const [state, setState] = useState(initialState);
```

JSX

```jsx
import { useState } from 'react';

function Counter() {
    const [count, setCount] = useState(0);

    return (
        <div>
            <p>Has hecho clic {count} veces</p>
            <button onClick={() => setCount(count + 1)}>Aumentar</button>
        </div>
    );
}
```

Why Is State Asynchronous?

- **Batching** — React groups multiple state updates into a single re-render for performance.
- **Consistency** — Ensures state is consistent across the component and its children.

Handling Async State Updates

Using a callback in setState:

```JSX
const handleClick = () => {
    setCount((prevCount) => prevCount + 1);
    console.log(count); // Still logs the old value
};
```

Using useEffect:

```JSX
import React, { useState, useEffect } from 'react';

function Counter() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        console.log('Count updated:', count);
    }, [count]);

    const handleClick = () => {
        setCount(count + 1);
    };

    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={handleClick}>Increment</button>
        </div>
    );
}
```

useReducer

Ideal for managing complex state logic with multiple sub-values. Follows the Redux pattern where state updates are handled by a reducer function.

```JSX
const [state, dispatch] = useReducer(reducer, initialState, init?);
```

- **reducer** — A pure function that takes the current state and an action, returns the new state.

- **dispatch** — A function that sends actions to the reducer.
- **initialState** — The initial state value.
- **init** (optional) — A function to lazily initialize state.

Reducer Rules

- Must be a pure function (no side effects, no async, no localStorage).
- Must always return a new state.
- Should only require an action, which may have a payload.

Example

JSX

```jsx
import React, { useReducer } from 'react';

const initialState = { count: 0 };

function init(initialValue) {
    return { count: initialValue * 2 };
}

function reducer(state, action) {
    switch (action.type) {
        case 'increment':
            return { count: state.count + 1 };
        case 'decrement':
            return { count: state.count - 1 };
        case 'reset':
            return init(action.payload);
        default:
            throw new Error(`Unsupported action: ${action.type}`);
    }
}

function Counter() {
    const [state, dispatch] = useReducer(reducer, 5, init);

    return (
        <div>
            <h1>Count: {state.count}</h1>
            <button onClick={() => dispatch({ type: 'increment' })}>Incre
            <button onClick={() => dispatch({ type: 'decrement' })}>Decre
            <button onClick={() => dispatch({ type: 'reset', payload: 5 }
        </div>
    );
}

export default Counter;
```

> **Important:** If you call the reducer directly in each component instead of passing state/dispatch as props, you'd create a new reducer instance per component.

React Hooks: Effects

useEffect

Performs side effects in functional components: data fetching, event subscriptions, DOM manipulation, timers.

Combines `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` from class components.

When Does useEffect Run?

After every render (no dependency array):

```jsx
useEffect(() => {
    // Runs on every render
});
```

Only once (empty dependency array):

```jsx
useEffect(() => {
    // Runs only on the first render
}, []);
```

When dependencies change:

```jsx
useEffect(() => {
    // Runs on first render and whenever prop or state changes
}, [prop, state]);
```

Cleanup

Return a function from the effect. It runs before the effect re-runs and when the component unmounts. Prevents memory leaks.

JSX

```jsx
useEffect(() => {
    // Setup
    return () => {
        // Cleanup
    };
}, []);
```

Common Use Cases

Aborting Fetch Requests:

JSX

```jsx
import React, { useState, useEffect } from 'react';

function DataFetcher({ userId }) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        const abortController = new AbortController();

        const fetchData = async () => {
            try {
                const response = await fetch(`https://api.example.com/use
                    signal: abortController.signal,
                });
                const result = await response.json();
                setData(result);
                setLoading(false);
            } catch (error) {
                if (error.name === 'AbortError') {
                    console.log('Fetch aborted');
                } else {
                    setError(error);
                    setLoading(false);
                }
            }
        };

        fetchData();

        return () => {
            abortController.abort();
        };
    }, [userId]);

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error.message}</p>;

    return (
        <div>
            <h1>User Data:</h1>
            <pre>{JSON.stringify(data, null, 2)}</pre>
        </div>
    );
}
```

Cleanup of setInterval:

JSX

```jsx
import React, { useState, useEffect } from "react";

function Timer() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            setCount((prev) => prev + 1);
        }, 1000);

        return () => clearInterval(interval);
    }, []);

    return <div>Count: {count}</div>;
}

export default Timer;
```

Cleanup of DOM event:

JSX

```jsx
import React, { useEffect } from "react";

function ResizeListener() {
    useEffect(() => {
        const handleResize = () => {
            console.log("Window resized!");
        };

        window.addEventListener("resize", handleResize);

        return () => {
            window.removeEventListener("resize", handleResize);
        };
    }, []);

    return <div>Resize the window and check the console!</div>;
}

export default ResizeListener;
```

> **Note:** Without cleanup, `getEventListeners(window)` in the browser console would show stale subscriptions, causing memory leaks.

Cleanup of WebSocket subscription:

JSX

```jsx
import React, { useEffect } from "react";

function WebSocketComponent() {
    useEffect(() => {
        const socket = new WebSocket("wss://example.com/socket");

        socket.onmessage = (event) => {
            console.log("Message from server:", event.data);
        };

        return () => {
            socket.close();
        };
    }, []);

    return <div>WebSocket is running...</div>;
}

export default WebSocketComponent;
```

useLayoutEffect

Fires **synchronously** after DOM mutations but **before** the browser repaints. Use it when you need to measure or modify the DOM and prevent visual flicker.

JSX

```jsx
import { useLayoutEffect, useRef, useState } from 'react';

function Tooltip() {
    const ref = useRef(null);
    const [tooltipHeight, setTooltipHeight] = useState(0);

    useLayoutEffect(() => {
        const { height } = ref.current.getBoundingClientRect();
        setTooltipHeight(height);
    }, []);

    return (
        <div>
            <div ref={ref} className="tooltip">
                I'm a tooltip
            </div>
            <p>Tooltip height: {tooltipHeight}px</p>
        </div>
    );
}
```

useEffect vs useLayoutEffect

	useEffect	useLayoutEffect
Timing	After paint	Before paint
Blocking	No	Yes (blocks rendering)
Use for	Data fetching, subscriptions, general side effects	DOM measurements, preventing flicker

> Almost always prefer `useEffect`. Only use `useLayoutEffect` when you need to measure or modify the DOM synchronously before the browser paints.

React Hooks: Refs & Memoization

useRef

Creates a mutable object with a `.current` property that persists across renders. Updating `.current` does **not** trigger a re-render.

```jsx
const ref = useRef(initialValue);
```

Accessing DOM Elements

```jsx
import { useRef } from 'react';

function FocusExample() {
    const inputRef = useRef(null);

    const focusInput = () => {
        inputRef.current.focus();
    };

    return (
        <div>
            <input ref={inputRef} type="text" placeholder="Escribe algo..
            <button onClick={focusInput}>Focus Input</button>
        </div>
    );
}
```

Traditional JS vs React

Traditional JavaScript:

JS

```
const inputRef = document.getElementById('myInput');
const focusButton = document.getElementById('focusButton');

const handleFocus = function () {
    inputRef.focus();
};

focusButton.addEventListener('click', handleFocus);
```

React with useRef:

JSX

```
import { useRef } from 'react';

const FocusComponent = () => {
    const inputRef = useRef(null);

    const handleFocus = () => {
        let inputElement = inputRef.current;
        inputElement.focus();
    };

    return (
        <div>
            <input type="text" ref={inputRef} />
            <button onClick={handleFocus}>Focus Input</button>
        </div>
    );
};
```

Storing Mutable Values

`useRef` can store any value — not just DOM elements. Unlike `useState`, updating a ref does **not** re-render the component. This makes it useful for values that change over time but don't affect the UI directly.

JSX

```jsx
import { useRef, useState } from 'react';

function Timer() {
    const [count, setCount] = useState(0);
    const intervalRef = useRef(null);

    const startTimer = () => {
        intervalRef.current = setInterval(() => {
            setCount((prev) => prev + 1);
        }, 1000);
    };

    const stopTimer = () => {
        clearInterval(intervalRef.current);
    };

    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={startTimer}>Start</button>
            <button onClick={stopTimer}>Stop</button>
        </div>
    );
}
```

Without `useRef`, you'd need `useState` for the interval ID — which would trigger an unnecessary re-render every time the timer starts/stops.

useRef vs useState

	`useRef`	`useState`
Triggers re-render	No	Yes
Use case	Storing values that don't affect UI	Storing values that affect UI
Example	Timer ID, previous value, DOM element	Counter, form input, toggle

Common Use Cases

1. **Accessing DOM elements** — focus, measure dimensions, scroll
2. **Storing timer/interval IDs** — `setInterval`, `setTimeout`

3. **Tracking previous values** — compare current vs last prop/state
4. **Storing mutable values** — values that change but shouldn't re-render
5. **Keeping references to third-party instances** — maps, charts, WebSocket connections

useCallback

Memoizes a function so it only changes when its dependencies change. Prevents unnecessary re-renders of child components that receive the function as a prop.

```jsx
const memoizedCallback = useCallback(() => {
    // Function logic
}, [dependencies]);
```

Example:

JSX

```jsx
import React, { useState, useCallback } from 'react';

function TodoList({ tasks, addTask }) {
    console.log('TodoList renderizado');
    return (
        <div>
            <ul>
                {tasks.map((task, index) => (
                    <li key={index}>{task}</li>
                ))}
            </ul>
            <button onClick={addTask}>Add task</button>
        </div>
    );
}

function App() {
    const [tasks, setTasks] = useState(['Task 1', 'Task 2']);
    const [input, setInput] = useState('');

    const addTask = useCallback(() => {
        setTasks((prevTasks) => [...prevTasks, input]);
        setInput('');
    }, [input]);

    return (
        <div>
            <input
                type="text"
                value={input}
                onChange={(e) => setInput(e.target.value)}
                placeholder="New task"
            />
            <TodoList tasks={tasks} addTask={addTask} />
        </div>
    );
}

export default App;
```

> For a full breakdown of when to use `React.memo`, `useMemo`, and `useCallback` together, see: 09-performance-memoization.md

useMemo

Memoizes a computed value. Recalculates only when dependencies change. Useful for expensive computations or preventing unnecessary recalculations.

```jsx
const memoizedValue = useMemo(() => {
    return expensiveComputation(a, b);
}, [a, b]);
```

Example:

```jsx
import React, { useMemo, useState } from 'react';

function FilteredList({ items, filter }) {
    const [search, setSearch] = useState('');

    const filteredItems = useMemo(() => {
        console.log('Filtering...');
        return items
            .filter((item) => item.includes(filter))
            .filter((item) => item.includes(search));
    }, [items, filter, search]);

    return (
        <div>
            <input value={search} onChange={(e) => setSearch(e.target.val
            <p>Results: {filteredItems.length}</p>
        </div>
    );
}
```

Without `useMemo`, filtering runs on every render. With it, only when `items`, `filter`, or `search` change.

> `useMemo` returns the **value**, `useCallback` returns the **function**. They solve the same problem from different angles.

React Hooks: Advanced

Custom Hooks

JavaScript functions whose name starts with `use` and that can call other hooks. They encapsulate reusable logic.

useFetch Example

JSX

```jsx
import { useState, useEffect } from 'react';

function useFetch(url) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        const fetchData = async () => {
            try {
                const response = await fetch(url);
                if (!response.ok) {
                    throw new Error('Error fetching data');
                }
                const result = await response.json();
                setData(result);
            } catch (err) {
                setError(err.message);
            } finally {
                setLoading(false);
            }
        };

        fetchData();
    }, [url]);

    return { data, loading, error };
}

import React from 'react';
import useFetch from './useFetch';

function DataComponent() {
    const { data, loading, error } = useFetch('https://jsonplaceholder.ty

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error}</p>;

    return (
        <ul>
            {data.map((post) => (
                <li key={post.id}>{post.title}</li>
            ))}
        </ul>
    );
}
```

```
export default DataComponent;
```

useContext

Allows consuming a React Context value inside a functional component. Avoids prop drilling by letting deep children access data from an ancestor provider.

JSX

```jsx
import React, { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

const Button = () => {
    const { theme } = useContext(ThemeContext);
    return <button className={theme}>Click me</button>;
};
```

Returns whatever value was passed to the nearest `Provider` above in the tree. When the provider's value changes, all consumers re-render.

For the full Context pattern (create, provide, multiple contexts, performance): see Context API.

React Context API

What is Context?

Context provides a way to pass data through the component tree without having to pass props manually at every level. It solves the **prop drilling** problem — when you need to pass data through many intermediate components that don't use it.

> Lifting State Up — sharing state by moving it to the closest common parent.

TEXT

```
// Without Context: prop drilling
App (theme="dark")
 └─ Layout (theme="dark") ← doesn't use it
     └─ Sidebar (theme="dark") ← doesn't use it
         └─ Button (theme="dark") ← finally uses it

// With Context: direct access
App (ThemeContext.Provider value="dark")
 └─ Layout
     └─ Sidebar
         └─ Button (useContext(ThemeContext)) ← reads directly
```

When to Use Context vs Props

Use Props	Use Context
Data needed by 1-2 levels	Data needed by 3+ levels
Component-specific data	Global/shared data (theme, auth, locale)
Simple data passing	State that many components read

Rule of thumb: If you're passing the same prop through 2+ intermediate components, consider Context.

Creating Context

JSX

```jsx
import { createContext } from 'react';

const ThemeContext = createContext('light'); // default value
```

JSX

```jsx
import { createContext, useContext } from 'react';

// 1️⃣ Crear el contexto con valor por defecto
const ThemeContext = createContext('light');

// 2️⃣ Componente que CONSUME el contexto
function Button() {
    const theme = useContext(ThemeContext);
    return <button>El tema es: {theme}</button>;
}

// 3️⃣ CASO A: SIN PROVIDER
function App() {
    return (
        <div>
            <Button />  {/* ← No hay Provider arriba */}
        </div>
    );
}
// Resultado: "El tema es: light" (valor por defecto)

// 4️⃣ CASO B: CON PROVIDER
function App() {
    return (
        <ThemeContext.Provider value="dark">
            <Button />  {/* ← Está DENTRO del Provider */}
        </ThemeContext.Provider>
    );
}
// Resultado: "El tema es: dark" (valor del Provider)
```

> In practice, most apps use `createContext(null)` — this forces you to always wrap with a Provider. If someone forgets, they get `null` immediately — easier to debug than a silent wrong value.

Provider

Wraps the part of the tree that needs access to the context value. Any descendant can consume it.

JSX

```jsx
import React, { createContext, useState } from 'react';

// 1️⃣ Crear el contexto
const ThemeContext = createContext(null);

// 2️⃣ Provider: envuelve el árbol y provee el valor
export const ThemeProvider = ({ children }) => {
    const [theme, setTheme] = useState('light');

    const toggleTheme = () => {
        setTheme((prev) => (prev === 'light' ? 'dark' : 'light'));
    };

    return (
        <ThemeContext.Provider value={{ theme, toggleTheme }}>
            {children}
        </ThemeContext.Provider>
    );
};
```

Usage in App:

JSX

```jsx
import { ThemeProvider } from './ThemeContext';

function App() {
    return (
        <ThemeProvider>
            <Layout />
        </ThemeProvider>
    );
}
```

Consuming with useContext

JSX

```jsx
import React, { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

const Header = () => {
    const { theme, toggleTheme } = useContext(ThemeContext);

    return (
        <header className={theme}>
            <p>Current theme: {theme}</p>
            <button onClick={toggleTheme}>Toggle</button>
        </header>
    );
};
```

`useContext` reads the nearest `Provider` value above the component. When the provider's value changes, all consumers re-render.

Multiple Contexts

Combine independent contexts to avoid unnecessary re-renders. Each context triggers re-renders only for its own consumers.

JSX

```jsx
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext();
const AuthContext = createContext();

function App() {
    const [theme, setTheme] = useState('light');
    const [user, setUser] = useState(null);

    return (
        <AuthContext.Provider value={{ user, setUser }}>
            <ThemeContext.Provider value={{ theme, setTheme }}>
                <Layout />
            </ThemeContext.Provider>
        </AuthContext.Provider>
    );
}

// Each component picks only what it needs
const Header = () => {
    const { theme } = useContext(ThemeContext);
    const { user } = useContext(AuthContext);
    return <header className={theme}>Welcome, {user?.name}</header>;
};
```

Why separate? If theme and auth are in the same context, changing the theme re-renders components that only care about auth. Separate contexts = granular re-renders.

Performance Considerations

Problem: Context + useMemo

When the Provider's `value` changes, **every consumer** re-renders — even if the component only uses a subset of the value.

JSX

```jsx
// BAD: new object every render → all consumers re-render
<ThemeContext.Provider value={{ theme, toggleTheme }}>
```

Solution 1: useMemo in the Provider:

```jsx
const contextValue = useMemo(
    () => ({ theme, toggleTheme }),
    [theme, toggleTheme]
);

<ThemeContext.Provider value={contextValue}>
```

Solution 2: Split contexts (as shown above).

Solution 3: React.memo on consumers:

```jsx
const Header = React.memo(() => {
    const { theme } = useContext(ThemeContext);
    return <header className={theme}>Hello</header>;
});
```

This works because `useContext` inside a memo'd component still triggers re-renders when the context value changes, but `React.memo` prevents re-renders from parent prop changes.

When Context Causes Unnecessary Re-renders

```jsx
// BAD: value changes on every render (new array created)
<AuthContext.Provider value={[user, setUser]}>

// GOOD: stable reference
const authValue = useMemo(() => ({ user, setUser }), [user]);
<AuthContext.Provider value={authValue}>
```

Best Practices

1. **Don't put everything in one context.** Split by concern (theme, auth, locale).
2. **Keep Provider value stable.** Use `useMemo` or `useCallback` to avoid unnecessary re-renders.
3. **Default values are for testing.** In production, always wrap with a Provider.
4. **Don't overuse Context.** For component-specific state (form inputs, modals), local `useState` is better.
5. **Avoid nesting Providers deeply.** It makes the component tree harder to read. Consider combining related providers into a single wrapper.

JSX

```jsx
// Clean: compose providers
function AppProviders({ children }) {
    return (
        <AuthProvider>
            <ThemeProvider>
                <RouterProvider>
                    {children}
                </RouterProvider>
            </ThemeProvider>
        </AuthProvider>
    );
}

// Usage
function App() {
    return <AppProviders><Layout /></AppProviders>;
}
```

Quick Reference

Concept	API
Create	`const Ctx = createContext(default)`
Provide	`<Ctx.Provider value={...}>`
Consume	`useContext(Ctx)`
Stable value	`useMemo(() ⇒ ({ ... }), [deps])`

> For `useContext` hook basics, see: 06-hooks.md

Forms Patterns: Controlled vs Uncontrolled

Intro

React has two ways to handle form inputs:

- **Controlled:** React manages the input state. The value lives in `useState` and updates via `onChange`.
- **Uncontrolled:** The DOM manages the input state. A `ref` reads the value when needed.

The difference is **who owns the data**: React or the browser.

```text
Controlled:
  User types → onChange fires → setState → re-render → input shows new va

Uncontrolled:
  User types → DOM updates directly → ref reads value on demand
```

Controlled Components

The input's `value` is bound to React state. Every keystroke triggers `onChange`, which updates state, which re-renders the input with the new value.

JSX

```jsx
import React, { useState } from 'react';

function SearchBar() {
    const [query, setQuery] = useState('');

    const handleChange = (event) => {
        setQuery(event.target.value);
    };

    return (
        <div>
            <input type="text" value={query} onChange={handleChange} />
            <p>You searched: {query}</p>
        </div>
    );
}
```

How it works:

- `value={query}` — input always shows the state value
- `onChange={handleChange}` — updates state on every keystroke
- React "controls" the input — the DOM never diverges from React state

value vs defaultValue

Prop	When	Use case
`value`	Controlled — value always comes from state	Search bars, dynamic forms
`defaultValue`	Uncontrolled — sets initial value, then DOM owns it	Simple forms, edit-once fields

JSX

```jsx
// Controlled: value is always controlled by React
<input value={name} onChange={(e) => setName(e.target.value)} />

// Uncontrolled: defaultValue sets the initial value, DOM owns it after
<input defaultValue="John" ref={inputRef} />
```

> **Gotcha:** Using `value` without `onChange` makes the input read-only. React warns you: "You provided a `value` prop without an `onChange` handler."

Real-time Validation

Controlled components make validation easy — you can check the value on every keystroke:

JSX

```jsx
function EmailInput() {
    const [email, setEmail] = useState('');
    const [error, setError] = useState('');

    const handleChange = (e) => {
        const value = e.target.value;
        setEmail(value);

        if (!value.includes('@')) {
            setError('Invalid email');
        } else {
            setError('');
        }
    };

    return (
        <div>
            <input type="email" value={email} onChange={handleChange} />
            {error && <p style={{ color: 'red' }}>{error}</p>}
        </div>
    );
}
```

Uncontrolled Components

The input manages its own state. A `ref` reads the value when you need it (e.g., on form submission).

JSX

```jsx
import React, { useRef } from 'react';

function SimpleForm() {
    const nameRef = useRef(null);

    const handleSubmit = (event) => {
        event.preventDefault();
        alert(`Hello, ${nameRef.current.value}`);
    };

    return (
        <form onSubmit={handleSubmit}>
            <input type="text" ref={nameRef} defaultValue="John" />
            <button type="submit">Submit</button>
        </form>
    );
}
```

How it works:

- `ref={nameRef}` — connects the input to a ref object
- `defaultValue="John"` — sets the initial value, DOM owns it after
- `nameRef.current.value` — reads the current DOM value on submit

When Refs Are Useful

- File inputs (`<input type="file">`) — always uncontrolled, no `value` prop
- Integrating with non-React code
- Focusing an input programmatically

JSX

```jsx
function FileUpload() {
    const fileRef = useRef(null);

    const handleSubmit = (e) => {
        e.preventDefault();
        const file = fileRef.current.files[0];
        console.log(file.name);
    };

    return (
        <form onSubmit={handleSubmit}>
            <input type="file" ref={fileRef} />
            <button>Upload</button>
        </form>
    );
}
```

Multiple Inputs Pattern

For forms with many fields, use a single handler with a dynamic state key:

JSX

```jsx
function RegistrationForm() {
    const [form, setForm] = useState({
        name: '',
        email: '',
        password: '',
    });

    const handleChange = (e) => {
        const { name, value } = e.target;
        setForm((prev) => ({ ...prev, [name]: value }));
    };

    const handleSubmit = (e) => {
        e.preventDefault();
        console.log(form);
    };

    return (
        <form onSubmit={handleSubmit}>
            <input name="name" value={form.name} onChange={handleChange}
            <input name="email" value={form.email} onChange={handleChange
            <input name="password" type="password" value={form.password}
            <button>Register</button>
        </form>
    );
}
```

Key points:

- Each input has a `name` attribute matching the state key
- `[name]: value` dynamically updates the correct field
- One handler for all inputs — no need for `handleChangeName`, `handleChangeEmail`, etc.

Form Submission

	Controlled	Uncontrolled
Read values	From state (`form.email`)	From refs (`emailRef.current.value`)
Validate	On every keystroke or on submit	On submit only
Reset	`setForm(initialState)`	`formRef.current.reset()`
Submit handler	`onSubmit` reads state	`onSubmit` reads refs

JSX

```jsx
// Controlled: reset with state
function ControlledForm() {
    const [form, setForm] = useState({ name: '', email: '' });

    const handleSubmit = (e) => {
        e.preventDefault();
        console.log(form);
        setForm({ name: '', email: '' }); // reset
    };

    // ...
}

// Uncontrolled: reset with DOM method
function UncontrolledForm() {
    const formRef = useRef(null);

    const handleSubmit = (e) => {
        e.preventDefault();
        const formData = new FormData(formRef.current);
        console.log(Object.fromEntries(formData));
        formRef.current.reset(); // reset
    };

    return (
        <form ref={formRef} onSubmit={handleSubmit}>
            <input name="name" />
            <input name="email" />
            <button>Submit</button>
        </form>
    );
}
```

Table: Controlled vs Uncontrolled

Feature	Controlled	Uncontrolled
State owner	React (`useState`)	DOM
Value access	`state.value`	`ref.current.value`
Updates	`onChange` → `setState`	DOM updates directly
Initial value	`value` prop	`defaultValue` prop
Validation	Real-time (every keystroke)	On submit
Reset	`setState(initial)`	`formRef.reset()`
Re-renders	On every keystroke	Fewer (DOM handles updates)
Code complexity	More verbose	Less code
Best for	Complex forms, live validation, search	Simple forms, file inputs, quick prototyping

When to Use Each

Use Controlled when:

- You need real-time validation or feedback
- You need to dynamically disable/enable submit buttons
- You need to populate fields programmatically (e.g., autofill)
- You're building complex multi-step forms

Use Uncontrolled when:

- The form is simple and you only need values on submit
- You're integrating with non-React code
- You're working with file inputs
- Performance is critical and you want to avoid re-renders on every keystroke

In practice: Most React apps use controlled components. They're more predictable and easier to debug. Use uncontrolled only when you have a specific reason.

Performance & Memoization

React re-renders a component when its **state or props change**. This means when a parent re-renders, all its children re-render too — even if their props haven't changed. Memoization tools prevent these unnecessary re-renders.

The Problem: Unnecessary Re-renders

JSX

```jsx
function Parent() {
    const [count, setCount] = useState(0);

    return (
        <div>
            <button onClick={() => setCount(count + 1)}>Count: {count}</b
            <ExpensiveChild />
        </div>
    );
}

function ExpensiveChild() {
    console.log('ExpensiveChild rendered');
    // Heavy rendering work...
    return <p>I'm a heavy component</p>;
}
```

Every time `count` changes, `ExpensiveChild` re-renders too — even though it receives no props and its output is always the same. In small apps this is fine, but in complex UIs with expensive components it causes noticeable lag.

React.memo

Wraps a component to skip re-rendering when its props haven't changed (shallow comparison).

JSX

```jsx
const ExpensiveChild = React.memo(function ExpensiveChild() {
    console.log('ExpensiveChild rendered');
    return <p>I'm a heavy component</p>;
});
```

Now `ExpensiveChild` only re-renders when its props actually change. See React.memo for full details.

useMemo for Derived Props

When you compute a value in the parent and pass it as a prop, it creates a new reference every render unless you memoize it.

JSX

```jsx
function Parent({ items }) {
    const [query, setQuery] = useState('');

    // New array reference every render, even if items didn't change
    const filtered = items.filter((i) => i.name.includes(query));

    return (
        <div>
            <input value={query} onChange={(e) => setQuery(e.target.value
            <MemoizedList items={filtered} />
        </div>
    );
}

const MemoizedList = React.memo(function MemoizedList({ items }) {
    console.log('MemoizedList rendered');
    return (
        <ul>
            {items.map((i) => (
                <li key={i.id}>{i.name}</li>
            ))}
        </ul>
    );
});
```

`filtered` is a new array on every render → `MemoizedList` always re-renders. Fix:

JSX

```jsx
const filtered = useMemo(
    () => items.filter((i) => i.name.includes(query)),
    [items, query]
);
```

Now `filtered` only changes when `items` or `query` change.

Why Memoize Functions?

This is the key concept most developers miss. Consider this:

JSX

```jsx
function Parent() {
    const [count, setCount] = useState(0);

    const handleClick = () => {
        console.log('clicked');
    };

    return (
        <div>
            <button onClick={() => setCount(count + 1)}>Count: {count}</b
            <MemoizedChild onClick={handleClick} />
        </div>
    );
}

const MemoizedChild = React.memo(function MemoizedChild({ onClick }) {
    console.log('MemoizedChild rendered');
    return <button onClick={onClick}>Click me</button>;
});
```

`MemoizedChild` **re-renders on every parent render.** Why?

The Problem

`handleClick` is recreated as a **new function** on every render of `Parent`. Even though the function body is identical, it's a **different reference in memory** each time. `React.memo` compares props by reference — new function = new prop = re-render.

TEXT

```
Render 1: handleClick → function at 0x001
Render 2: handleClick → function at 0x023  (new reference!)
React.memo sees: onClick changed → re-render
```

The Solution: useCallback

JSX

```jsx
const handleClick = useCallback(() => {
    console.log('clicked');
}, []);
```

`useCallback` returns the **same function reference** across renders (as long as dependencies don't change). Now `React.memo` sees the same `onClick` prop and

skips the re-render.

```text
Render 1: handleClick → function at 0x001
Render 2: handleClick → function at 0x001  (same reference)
React.memo sees: onClick didn't change → skip re-render
```

Side-by-Side

```jsx
// Without useCallback — new reference every render
const handleClick = () => {
    console.log('clicked');
};

// With useCallback — same reference across renders
const handleClick = useCallback(() => {
    console.log('clicked');
}, []);
```

> **Without `React.memo` on the child, `useCallback` alone has no effect.** It only matters when the function is passed as a prop to a memoized component.

Internal Relationship

`useCallback` is syntactic sugar over `useMemo`:

```jsx
// These are equivalent:
const handleClick = useCallback(() => doSomething(), [dep]);
const handleClick = useMemo(() => () => doSomething(), [dep]);
```

`useMemo` memoizes the **return value**. `useCallback` memoizes the **function itself**.

The Combo Rule

`useCallback` and `React.memo` work together — but neither works alone:

Combination	Result
`React.memo` without `useCallback`	Child skips re-render when props don't change, but functions create new references every time — so it re-renders anyway
`useCallback` without `React.memo`	Function has stable reference, but the child re-renders regardless — memoization is wasted
`React.memo` + `useCallback`	Child skips re-render AND function reference is stable — full optimization

Example:

JSX

```jsx
function Parent() {
    const [count, setCount] = useState(0);

    // Without useCallback — new reference every render
    const handleClick = () => console.log('clicked');

    // With useCallback — same reference across renders
    const handleClickStable = useCallback(() => {
        console.log('clicked');
    }, []);

    return (
        <div>
            <button onClick={() => setCount(count + 1)}>Count: {count}</b
            {/* ❌ Without React.memo — re-renders anyway, useCallback do
            <Button onClick={handleClickStable} />

            {/* ✅ With React.memo — skips re-render */}
            <MemoizedButton onClick={handleClickStable} />
        </div>
    );
}

// Without React.memo — always re-renders
const Button = ({ onClick }) => {
    console.log('Button rendered');
    return <button onClick={onClick}>Click</button>;
};

// With React.memo — only re-renders when onClick changes
const MemoizedButton = React.memo(({ onClick }) => {
    console.log('MemoizedButton rendered');
    return <button onClick={onClick}>Click</button>;
});
```

Result:

- `Button` → re-renders every time `count` changes, even with `useCallback`
- `MemoizedButton` → skips re-render because `handleClickStable` has the same reference

Rule: If you memoize a function, make sure the component receiving it is wrapped in `React.memo`. Otherwise, don't bother with `useCallback`.

Without a Memoized Child

`useMemo` and `useCallback` are still useful even when you're NOT passing values to a child component.

useMemo: Expensive Computations

Avoid recalculating expensive operations on every render:

JSX

```jsx
function Dashboard({ data }) {
    // Without useMemo — recalculates on every render
    const stats = calculateStats(data);

    // With useMemo — only recalculates when data changes
    const stats = useMemo(() => calculateStats(data), [data]);

    return <div>Average: {stats.average}</div>;
}
```

useCallback: Stabilizing useEffect Dependencies

When a function is a dependency of `useEffect`, you need a stable reference to avoid infinite loops:

JSX

```jsx
function UserProfile({ userId }) {
    // Without useCallback — new function every render → useEffect runs e
    const fetchUser = async () => {
        const res = await fetch(`/api/users/${userId}`);
        // ...
    };

    // With useCallback — same reference → useEffect only runs when userI
    const fetchUser = useCallback(async () => {
        const res = await fetch(`/api/users/${userId}`);
        // ...
    }, [userId]);

    useEffect(() => {
        fetchUser();
    }, [fetchUser]); // Without useCallback, this creates an infinite loo
}
```

> **Key takeaway:** `useMemo` and `useCallback` don't require `React.memo` on a child. They're also valuable for optimizing your own component's logic.

All Three Together

The complete pattern for preventing unnecessary re-renders:

JSX

```jsx
function Parent({ items }) {
    const [count, setCount] = useState(0);
    const [query, setQuery] = useState('');

    // Memoized value — same reference unless items/query change
    const filtered = useMemo(
        () => items.filter((i) => i.name.includes(query)),
        [items, query]
    );

    // Memoized function — same reference across renders
    const handleSelect = useCallback((id) => {
        console.log('selected', id);
    }, []);

    return (
        <div>
            <button onClick={() => setCount(count + 1)}>Count: {count}</b
            <input value={query} onChange={(e) => setQuery(e.target.value
            {/* Child only re-renders when filtered or handleSelect chang
            <MemoizedList items={filtered} onSelect={handleSelect} />
        </div>
    );
}

const MemoizedList = React.memo(function MemoizedList({ items, onSelect }
    return (
        <ul>
            {items.map((i) => (
                <li key={i.id} onClick={() => onSelect(i.id)}>
                    {i.name}
                </li>
            ))}
        </ul>
    );
});
```

When NOT to Optimize

- **Cheap computations** — `useMemo` and `useCallback` have overhead (comparing dependencies, storing previous results). For simple operations like `a + b`, just compute directly.

- **No memoized children** — `useCallback` without `React.memo` on the child does nothing useful.
- **Small component trees** — React is already fast. Only optimize when you see actual performance issues.

> Always profile before optimizing. Use **React DevTools Profiler** to identify what's actually slow, then apply memoization where it matters.

Summary

Tool	What it memoizes	When to use
`React.memo`	A **component's render**	Component is expensive and re-renders with unchanged props
`useMemo`	A **computed value**	Expensive calculation or value passed as prop to memoized child
`useCallback`	A **function reference**	Function passed as prop to memoized child

Rule of thumb: Start without any memoization. Profile. If a component or computation is slow, apply the appropriate tool.

React Router DOM

What is React Router?

React Router is a routing library for React that enables navigation between different views/pages in a single-page application (SPA) without full page reloads.

```text
npm install react-router-dom
```

To install a specific version:

```bash
npm install react-router-dom@6
```

Setup

BrowserRouter

Wraps your entire app and enables client-side routing. Use `HashRouter` if you need hash-based URLs (`/home#section`).

```jsx
import { BrowserRouter } from "react-router-dom";

function App() {
    return (
        <BrowserRouter>
            {/* Your routes go here */}
        </BrowserRouter>
    );
}
```

Routes and Route

`Route` defines which component renders for a specific path. `Routes` is the container that holds all your routes.

JSX

```jsx
import { Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";

function App() {
    return (
        <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/about" element={<About />} />
            <Route path="*" element={<NotFound />} />
        </Routes>
    );
}
```

- `path` — URL pattern to match
- `element` — Component to render when the path matches
- `*` — Catch-all route (404)

Navigation

Link

Creates navigation links without full page reloads. Prevents unnecessary re-renders.

JSX

```jsx
import { Link } from "react-router-dom";

function Navigation() {
    return (
        <nav>
            <Link to="/">Home</Link>
            <Link to="/about">About</Link>
        </nav>
    );
}
```

NavLink

Like `Link`, but adds an `isActive` class when the link matches the current route.

```jsx
import { NavLink } from "react-router-dom";

function Navigation() {
    return (
        <nav>
            <NavLink to="/">Home</NavLink>
            <NavLink to="/about">About</NavLink>
        </nav>
    );
}
```

useNavigate

Programmatic navigation between routes.

```jsx
import { useNavigate } from "react-router-dom";

function MyComponent() {
    const navigate = useNavigate();

    function handleClick() {
        navigate("/about");
    }

    return <button onClick={handleClick}>Go to About</button>;
}
```

With replace:

```jsx
navigate("/about", { replace: true }); // replaces current history entry
```

Go back:

```jsx
navigate(-1); // go back one step
```

Dynamic Routes

useParams

Access dynamic URL parameters (e.g., `/post/:id`).

```jsx
import { useParams } from "react-router-dom";

function Post() {
    const { id } = useParams();
    return <div>Post ID: {id}</div>;
}
```

useLocation

Returns the current URL location object.

```jsx
import { useLocation } from "react-router-dom";

function CurrentLocation() {
    const location = useLocation();
    return (
        <div>
            <p>Path: {location.pathname}</p>
            <p>Search: {location.search}</p>
            <p>Hash: {location.hash}</p>
        </div>
    );
}
```

useMatch

Check if the current location matches a given path.

```jsx
import { useMatch } from "react-router-dom";

function MyComponent() {
    const match = useMatch("/about");
    return match ? <div>You're on the About page</div> : null;
}
```

Nested Routes

Nested routes allow you to render child routes inside a parent layout. Use `Outlet` to define where child routes render.

JSX

```jsx
import { Routes, Route, Outlet } from "react-router-dom";

function Dashboard() {
    return (
        <div>
            <h1>Dashboard</h1>
            <nav>
                <Link to="profile">Profile</Link>
                <Link to="settings">Settings</Link>
            </nav>
            {/* Child routes render here */}
            <Outlet />
        </div>
    );
}

function Profile() {
    return <h2>Profile</h2>;
}

function Settings() {
    return <h2>Settings</h2>;
}

function App() {
    return (
        <Routes>
            <Route path="/dashboard" element={<Dashboard />}>
                <Route path="profile" element={<Profile />} />
                <Route path="settings" element={<Settings />} />
            </Route>
        </Routes>
    );
}
```

- `Dashboard` renders at `/dashboard`
- `Profile` renders at `/dashboard/profile` inside `Dashboard`
- `Settings` renders at `/dashboard/settings` inside `Dashboard`
- `<Outlet />` is where child routes appear

Protected Routes

Auth guard pattern — redirect to login if user is not authenticated.

JSX

```jsx
import { Navigate, Outlet } from "react-router-dom";

function ProtectedRoute({ isAuthenticated }) {
    if (!isAuthenticated) {
        return <Navigate to="/login" replace />;
    }
    return <Outlet />;
}

function App() {
    const [isAuthenticated, setIsAuthenticated] = useState(false);

    return (
        <Routes>
            <Route path="/login" element={<Login />} />
            <Route element={<ProtectedRoute isAuthenticated={isAuthentica
                <Route path="/dashboard" element={<Dashboard />} />
                <Route path="/profile" element={<Profile />} />
            </Route>
        </Routes>
    );
}
```

Redirects

Navigate Component

Redirects to another route. Useful for default routes or auth redirects.

JSX

```jsx
import { Navigate } from "react-router-dom";

function App() {
    return (
        <Routes>
            <Route path="/" element={<Navigate to="/home" replace />} />
            <Route path="/home" element={<Home />} />
            <Route path="/old-page" element={<Navigate to="/new-page" rep
        </Routes>
    );
}
```

Programmatic Redirect

JSX

```jsx
import { useNavigate } from "react-router-dom";

function Login() {
    const navigate = useNavigate();

    const handleLogin = async () => {
        await loginUser();
        navigate("/dashboard", { replace: true });
    };

    return <button onClick={handleLogin}>Login</button>;
}
```

Advanced

useRoutes

Define routes as an array of objects for more structured route management.

JSX

```jsx
import { BrowserRouter, useRoutes } from "react-router-dom";

function Home() { return <h2>Home Page</h2>; }
function About() { return <h2>About Page</h2>; }
function NotFound() { return <h2>404 - Not Found</h2>; }

function App() {
    const routes = [
        { path: "/", element: <Home /> },
        { path: "/about", element: <About /> },
        { path: "*", element: <NotFound /> },
    ];

    const element = useRoutes(routes);
    return element;
}

function Main() {
    return (
        <BrowserRouter>
            <App />
        </BrowserRouter>
    );
}

export default Main;
```

useSearchParams

Read and modify URL search parameters.

JSX

```jsx
import { useSearchParams } from "react-router-dom";

function SearchPage() {
    const [searchParams, setSearchParams] = useSearchParams();
    const query = searchParams.get("q") || "";

    return (
        <div>
            <input
                value={query}
                onChange={(e) => setSearchParams({ q: e.target.value })}
            />
            <p>Search: {query}</p>
        </div>
    );
}

// URL: /search?q=react
```

Full Example

JSX

```jsx
import React, { useState } from "react";
import {
    BrowserRouter,
    Routes,
    Route,
    Link,
    Navigate,
    Outlet,
    useParams,
} from "react-router-dom";

// Protected Route wrapper
function ProtectedRoute({ isAuthenticated }) {
    if (!isAuthenticated) return <Navigate to="/login" replace />;
    return <Outlet />;
}

// Layout
function Layout() {
    return (
        <div>
            <nav>
                <Link to="/">Home</Link> | <Link to="/dashboard">Dashboar
            </nav>
            <Outlet />
        </div>
    );
}

// Pages
function Home() { return <h2>Home</h2>; }
function Login() { return <h2>Login</h2>; }
function Dashboard() { return <h2>Dashboard</h2>; }
function Profile() {
    const { id } = useParams();
    return <h2>Profile: {id}</h2>;
}

function App() {
    const [isAuthenticated, setIsAuthenticated] = useState(false);

    return (
        <BrowserRouter>
            <Routes>
                <Route element={<Layout />}>
                    <Route path="/" element={<Home />} />
                    <Route path="/login" element={<Login />} />
```

```
                    <Route element={<ProtectedRoute isAuthenticated={isAu
                        <Route path="/dashboard" element={<Dashboard />}
                        <Route path="/profile/:id" element={<Profile />}
                    </Route>
                    <Route path="*" element={<Navigate to="/" replace />}
                </Route>
            </Routes>
        </BrowserRouter>
    );
}

export default App;
```

Quick Reference

Hook / Component	What it does
`Link` / `NavLink`	Navigation without page reload
`useNavigate`	Programmatic navigation
`useParams`	Access dynamic URL parameters
`useLocation`	Access current URL location
`useMatch`	Check if location matches a path
`useSearchParams`	Read/write URL search parameters
`useRoutes`	Define routes as objects
`Navigate`	Redirect to another route
`Outlet`	Render child routes in nested layouts

> **Tip:** When refreshing the page, app state (like authentication) is lost. Persist auth state in `localStorage` or `sessionStorage` to prevent unwanted redirects.

Class Components React

Creating a React App

TEXT

```
npx create-react-app my-app
cd my-app
npm start
```

The render() Method

Returns JSX elements describing the UI.

JSX

```jsx
import React, { Component } from 'react';

class App extends Component {
    render() {
        return (
            <div>
                <h1>Hello, World!</h1>
            </div>
        );
    }
}

export default App;
```

Props

Data passed from parent to child component via `this.props`.

JSX

```jsx
import React, { Component } from 'react';
import Greet from './components/Greet';

class App extends Component {
    render() {
        return (
            <div>
                <Greet name={'Alan'} />
            </div>
        );
    }
}

export default App;
```

JSX

```jsx
import React, { Component } from "react";

class Greet extends Component {
    render() {
        return <h1>Hello {this.props.name}</h1>;
    }
}

export default Greet;
```

State

Managed via `this.state` (initialized in constructor) and `this.setState()`.

JSX

```jsx
import { Component } from 'react';

class Counter extends Component {
    constructor(props) {
        super(props);
        this.state = {
            count: 0,
        };
    }

    incrementCount = () => {
        this.setState({ count: this.state.count + 1 });
    };

    render() {
        return (
            <div>
                <p>Count: {this.state.count}</p>
                <button onClick={this.incrementCount}>Increment</button>
            </div>
        );
    }
}

export default Counter;
```

Method Binding

Regular methods lose their `this` context when passed as callbacks. Arrow functions are the best choice for event handlers.

JSX

```jsx
// Regular method — loses context
toggleGoOut() {
    this.setState(prevState => ({
        goOut: prevState.goOut === "Yes" ? "No" : "Yes"
    }));
}

// Arrow function — preserves context
toggleGoOut = () => {
    this.setState(prevState => ({
        goOut: prevState.goOut === "Yes" ? "No" : "Yes"
    }));
};
```

Lifecycle Overview

Class components have three main phases:

Mounting	Updating	Unmounting
`constructor()`	`shouldComponentUpdate()`	`componentWillUnmount()`
`render()`	`render()`	
`componentDidMount()`	`componentDidUpdate()`	

Lifecycle: Mounting

Called when a component is first created and inserted into the DOM.

Method	When
`constructor()`	Initialize state, bind methods
`render()`	Return JSX
`componentDidMount()`	DOM is ready, fetch data, set up subscriptions

JSX

```jsx
class UserProfile extends Component {
    constructor(props) {
        super(props);
        this.state = { user: null };
    }

    componentDidMount() {
        fetch(`/api/users/${this.props.userId}`)
            .then((res) => res.json())
            .then((user) => this.setState({ user }));
    }

    render() {
        if (!this.state.user) return <p>Loading...</p>;
        return <h1>{this.state.user.name}</h1>;
    }
}
```

Equivalent in hooks:

JSX

```jsx
function UserProfile({ userId }) {
    const [user, setUser] = useState(null);

    useEffect(() => {
        fetch(`/api/users/${userId}`)
            .then((res) => res.json())
            .then(setUser);
    }, [userId]);

    if (!user) return <p>Loading...</p>;
    return <h1>{user.name}</h1>;
}
```

How the mapping works:

Class	Hooks	Why
`this.state = { user: null }`	`useState(null)`	Initial state
`componentDidMount`	`useEffect(() => {}, [])`	Empty array `[]` = run only once after first render
`this.setState({ user })`	`setUser(data)`	Update state
`this.props.userId`	`userId` (parameter)	Props become function parameters

> `useEffect` with `[]` runs once on mount — same as `componentDidMount`. If you add `[userId]`, it re-runs when `userId` changes.

Lifecycle: Updating

Called when props or state change.

Method	When
`shouldComponentUpdate()`	Before re-render, return `false` to skip
`render()`	Return updated JSX
`componentDidUpdate()`	After DOM update, side effects here

JSX

```jsx
class DataList extends Component {
    shouldComponentUpdate(nextProps) {
        // Only re-render if data actually changed
        return nextProps.data !== this.props.data;
    }

    componentDidUpdate(prevProps) {
        if (prevProps.data !== this.props.data) {
            console.log('Data changed, updating list...');
        }
    }

    render() {
        return (
            <ul>
                {this.props.data.map((item, i) => (
                    <li key={i}>{item}</li>
                ))}
            </ul>
        );
    }
}
```

Equivalent in hooks:

JSX

```jsx
function DataList({ data }) {
    useEffect(() => {
        console.log('Data changed, updating list...');
    }, [data]);

    return (
        <ul>
            {data.map((item, i) => (
                <li key={i}>{item}</li>
            ))}
        </ul>
    );
}

// React.memo prevents re-render if props haven't changed
export default React.memo(DataList);
```

How the mapping works:

Class	Hooks	Why
`shouldComponentUpdate(nextProps)`	`React.memo(Component)`	Both skip re-render if props haven't changed
`componentDidUpdate(prevProps)`	`useEffect(() => {}, [data])`	`data` in array = run only when `data` changes

> `React.memo` wraps the entire component — it compares all props automatically. `useEffect` with `[data]` runs the effect only when `data` changes, replacing the manual `prevProps` check.

Lifecycle: Unmounting

Called right before the component is removed from the DOM. Clean up subscriptions, timers, event listeners here.

JSX

```jsx
class Timer extends Component {
    constructor(props) {
        super(props);
        this.state = { seconds: 0 };
    }

    componentDidMount() {
        this.interval = setInterval(() => {
            this.setState((prev) => ({ seconds: prev.seconds + 1 }));
        }, 1000);
    }

    componentWillUnmount() {
        clearInterval(this.interval);
    }

    render() {
        return <p>Seconds: {this.state.seconds}</p>;
    }
}
```

Equivalent in hooks:

JSX

```jsx
function Timer() {
    const [seconds, setSeconds] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            setSeconds((prev) => prev + 1);
        }, 1000);

        return () => clearInterval(interval); // cleanup = componentWillU
    }, []);

    return <p>Seconds: {seconds}</p>;
}
```

How the mapping works:

Class	Hooks	Why
`componentDidMount`	`useEffect(() => { ... }, [])`	Start the timer once on mount
`componentWillUnmount`	`return () => { ... }`	The return function in `useEffect` runs on unmount
`this.interval = ...`	`const interval = ...`	Store the timer ID in a local variable

> In `useEffect`, the **return function** is the cleanup. It runs when the component is removed from the DOM — same as `componentWillUnmount`. The `[]` ensures the timer starts only once.

Error Boundaries

The **only** lifecycle that has no hook equivalent. Catches JavaScript errors in child components, displays a fallback UI instead of crashing the entire app.

JSX

```jsx
class ErrorBoundary extends Component {
    constructor(props) {
        super(props);
        this.state = { hasError: false };
    }

    static getDerivedStateFromError(error) {
        return { hasError: true };
    }

    componentDidCatch(error, errorInfo) {
        console.error('Error caught:', error, errorInfo);
    }

    render() {
        if (this.state.hasError) {
            return <h2>Something went wrong.</h2>;
        }
        return this.props.children;
    }
}
```

Usage:

JSX

```jsx
function App() {
    return (
        <ErrorBoundary>
            <BuggyComponent />
        </ErrorBoundary>
    );
}
```

> **Why no hook equivalent?** Error boundaries rely on the class lifecycle (`getDerivedStateFromError` + `componentDidCatch`). React has not added a hook alternative yet. This is one of the few valid reasons to use class components in new code.

Class vs Hooks

Feature	Class Component	Function + Hooks
State	`this.state` + `this.setState()`	`useState()`
Lifecycle	`componentDidMount`, etc.	`useEffect()`
Context	`static contextType`	`useContext()`
Refs	`this.refs` / `createRef()`	`useRef()`
Performance	No extra overhead	Slightly faster (no `this`)
Reusability	HOCs, render props	Custom hooks
Error handling	`componentDidCatch`	No equivalent
Code size	More boilerplate	Less verbose

When to Use Class Components

- **Legacy codebases:** Maintaining apps written with class components
- **Error boundaries:** The only case where class components are still required in new code
- **Third-party libraries:** Some APIs expose class-based component APIs

For everything else, prefer function components with hooks.